

RouteRec: Behavior-Guided Sparse Routing for Sequential Recommendation

Anonymous Author(s)

Abstract

Sequential recommendation models have improved recently by using stronger encoders and a richer behavioral context. However, they still rely on the same shared transformation for every session context, regardless of their different behavioral contexts and characteristics. Mixture of Experts (MoE), which has been one of the most successful techniques in language or vision domains, opens a different design choice: once conditional computation is available, what should define expert allocation in sessionized sequential recommendation? In this work, we propose RouteRec, a behavior-guided MoE framework for sequential recommendation. RouteRec creates behavioral cues from common signals existing in most datasets and uses them through coarse-to-fine multi-scope routing and hierarchical expert allocation. As a result, RouteRec uses information beyond the hidden states created from the backbone network and can be generally applied to most settings. Our experiments on six public datasets show that RouteRec outperforms nine strong competitors for sequential recommendation, achieving the best overall average rank of 1.67 compared to 3.56 for the next-best baseline. Our code is available at <https://anonymous.4open.science/r/RouteRec>.

Keywords

Sequential recommendation, sparse routing, conditional computation, behavioral cues, mixture of experts

ACM Reference Format:

Anonymous Author(s). 2026. RouteRec: Behavior-Guided Sparse Routing for Sequential Recommendation. In *Proceedings of the 20th ACM Conference on Recommender Systems (RecSys '26)*, September 28–October 2, 2026, Minneapolis, MN, USA. ACM, New York, NY, USA, 16 pages.

1 Introduction

Sequential recommendation aims to predict the next item from a user's interaction history [36]. Early methods focused on short-range transition patterns [29], while later models improved performance with stronger recurrent and self-attentive sequence encoders that can model longer and more complex dependencies [10, 16]. More recent work further strengthened these models with richer contextual signals, side information, and stronger training objectives, improving performance [5, 21, 24, 27, 31, 39, 42].

Despite this progress, most sequential recommendation models process all sessions or interactions using the same fixed computational path in the backbone architecture. This limits their performance, especially in session-based recommendation, where highly diverse characteristics. Some sessions repeat similar items, while others change quickly. Some stay focused, while others move across many options. These differences are clear in data, but most models still treat all sessions in the same way. Figure 1 gives a simple illustration: even sessions with clearly different behavioral patterns are

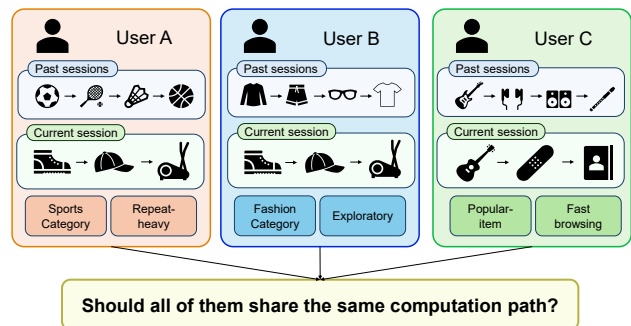


Figure 1: Three user's session sequences with different behavioral patterns receive the same shared transformation in standard sequential recommendation, while we aim to apply MoE for modeling their different behaviors.

still passed through the same shared transformation in standard sequential recommendation.

Mixture of Experts (MoE) offers a natural way to address this problem. MoE enables *conditional computation*, where only a data-dependent subset of experts is activated for each input [11, 15]. This means that different inputs can follow different computation paths. Once such routing is available, however, a new design question appears: if different sessions may benefit from different computation, what information should guide expert selection? Existing expert-based designs in sequential recommendation show that MoE can help, but routing is still often tied to hidden states, modality-specific inputs, or predefined preference factors [22, 25, 33].

We therefore ask a simple question: what kind of signal should a router use to separate sessions in a meaningful way? Our answer is to use *behavioral cues*, that is, compact signals that describe how a session behaves rather than only what items it contains. In sequential recommendation, sessions differ in pace, concentration, repetition, and exploration, and these differences matter for deciding which computation path should be used. This view is also consistent with how MoE has been used in other settings, where routing is often guided by clear differences already present in the problem, such as task identity, dataset identity, language identity, or user-specific priors [4, 12, 25, 33, 37]. Sequential recommendation raises the same design question, but here the relevant differences should be inferred from data, as they are not given explicitly.

Based on this idea, we propose **RouteRec**, which is our novel sequential recommendation model. RouteRec looks at common signals in sequential interaction data, such as how fast a session moves, how focused it is, how much it repeats, and how much it explores new items. From these signals, the model builds *behavioral cues*, which are simple indicators of how a session behaves. RouteRec then uses them to guide expert selection at different time ranges:

macro, mid, and micro, spanning from the interaction to the session level. In this way, it uses not only what happened in a session but also how the session behaves to decide *how to compute*.

In experiments, we compare RouteRec against nine strong baselines on six public benchmarks. RouteRec achieves the best overall average rank across datasets. Its improvements are largest when local next-step uncertainty is high, repetition patterns vary widely across sessions, and enough cross-session context is available. The advantage becomes smaller when these conditions are weak. Overall, these patterns are consistent with our view that routing should follow meaningful differences in session behavior.

Our main contributions are as follows:

- We frame MoE routing in sessionized sequential recommendation as a routing-design problem. We argue that expert selection should follow meaningful differences in session behavior, rather than relying only on hidden states.
- We propose **RouteRec**, a behavior-guided MoE framework that builds routing cues from common signals in sequential interaction data and uses them through coarse-to-fine multi-scale routing and hierarchical expert allocation.
- On six public datasets, RouteRec achieves the best overall average rank, improving the overall average rank from 3.56 for the strongest baseline to 1.67.

The rest of this paper is organized as follows. In Section 2, we introduce the problem definition and related work. In Section 3, we propose RouteRec, along with its main ideas, detailed implementation, and the objective function for its training. In Section 4, we provide experimental results comparing RouteRec with nine competitors. We conclude in Section 5. Our code and datasets are available at <https://anonymous.4open.science/r/RouteRec>.

2 Problem Definition and Related Work

2.1 Problem Definition

We consider session-based sequential recommendation as the target problem of this work. For each user, let $\mathcal{S}_u = [s_1, \dots, s_M]$ denote the chronologically ordered sequence of sessions, where each session $s_m = [x_{m,1}, \dots, x_{m,L_m}]$ contains items interacted with during one continuous activity window. The task is *next-item prediction*: given the t items $x_{m,1:t}$ in the current session and the user's past sessions $[s_1, \dots, s_{m-1}]$, we predict the next item $x_{m,t+1}$.

Each interaction $x_{m,t}$ is associated with an item identity $i_{m,t}$ and a timestamp $\tau_{m,t}$. In addition, each item i may have a coarse grouping label $c(i)$, such as category, genre, or artist, depending on the dataset. We also compute corpus-level popularity statistics $\pi(i)$ from the training set, where $\pi(i)$ denotes the frequency or relative popularity of item i in the observed training data. We use these observed fields as auxiliary signals for routing.

2.2 Sequential Recommendation

Sequential recommendation predicts the next item from ordered user interactions [36]. Session-based recommendation is a closely related setting that makes predictions from short-session context [13, 35]. Early models used Markov models [29], convolution-based architectures [34], and recurrent neural networks [10] to model

user behavior over time. Later work improved intent and transition modeling with attention and graph-based designs [20, 23, 38].

Transformer backbones such as SASRec and BERT4Rec have become strong model choices for sequential recommendation [16, 32]. More recent variants improve representation quality through contrastive objectives, frequency-aware bias, and state-space sequence modeling [5, 24, 27]. Side-information models such as FDSA and DIF-SR incorporate item attributes or timestamps to enrich the encoded representation [39, 42]. Across these lines of work, most models rely on a shared transformation for all inputs. Our work studies whether different sessions should be routed differently.

2.3 MoE-based Recommendation

MoE-based designs have already been explored in recommendation domains. In these models, experts are usually introduced to model a specific kind of variation in the data. M3SRec uses expert mixtures for multimodal representation learning in sequential recommendation [1]. HM4SR uses hierarchical experts to model time-related patterns in short-video recommendation [41]. FAME uses expert decomposition to model different preference facets in user behavior [22]. These works show that expert-based designs can help recommendation, but they are usually tied to specific modalities, temporal modules, or predefined preference factors. In contrast, RouteRec focuses more directly on the routing side: instead of starting from a fixed modality split or predefined facets, it asks what differences in session behavior should guide expert selection.

2.4 MoE Routing Design

A central question in MoE is how the router should choose experts. In many successful MoE models, expert selection is guided by clear differences that already matter in the problem. In multi-task recommendation, MMoE uses task-specific gates so that different tasks can combine experts differently [25]. PLE further separates shared experts from task-specific experts, allowing the model to capture both common patterns and task differences [33]. In multi-domain and multi-task recommendation, PEPNet uses personalized prior information to guide selection [4]. Similar ideas also appear outside recommendation. In visual understanding, DAMEX uses dataset-aware routing so that inputs from different datasets are sent to dataset-matched experts [12]. In multilingual speech recognition, LR-MoE uses frame-wise language routing to guide inputs to language-specific experts [37]. Across these works, routing is guided by explicit signals such as task, dataset, language, or user-specific priors, rather than being left fully implicit.

Our work brings this routing-design view to sequential recommendation, where such signals are less explicit. Instead of assuming task labels, dataset labels, language labels, or personalized priors, RouteRec builds behavioral cues directly from user interaction history. In this way, expert selection can follow differences in how sessions evolve over time, how focused they are, and how much they repeat or explore across different time ranges.

3 Proposed Method

We propose RouteRec, a behavior-guided MoE framework for sequential recommendation. RouteRec builds routing cues from common signals in sequential interaction data and uses them to guide

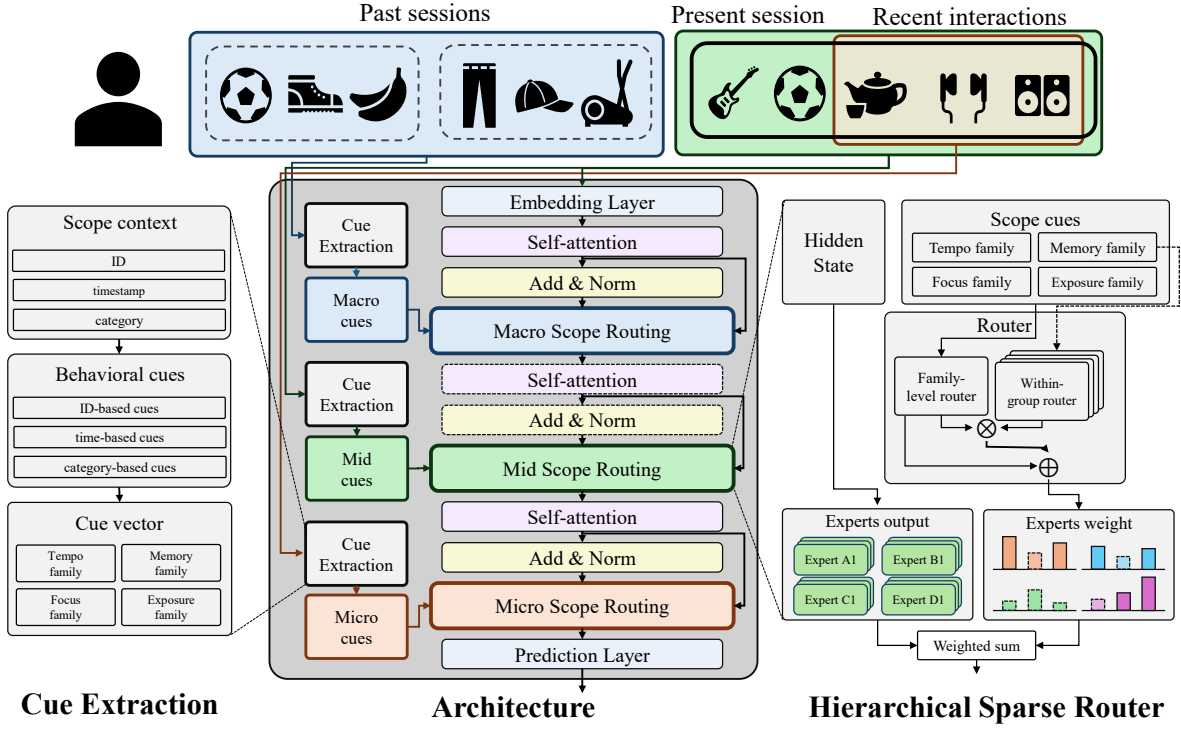


Figure 2: RouteRec overview. *Left:* raw interaction fields are summarized into four cue families under three temporal scopes. *Center:* each scope is instantiated by its own observation window and applied at a corresponding routing point between self-attention blocks, so that slower cross-session evidence and faster local evidence act at different points. *Right:* hierarchical sparse expert allocation first selects the most relevant cue families and then selects experts within those families to form the active computation path.

expert selection on top of a SASRec backbone. We first present an overview together with the three main ideas in Sec. 3.1, and then describe each component in detail in the following subsections.

3.1 Main Ideas and Architecture

RouteRec is built around three main ideas. Together, they define how routing cues are constructed, how they are used across different time ranges, and how experts are selected in a structured way.

Idea 1: Behavioral Cue Construction. RouteRec builds routing cues from simple signals that are available in most sequential recommendation datasets, including item identities, timestamps, optional coarse grouping labels, and training-derived popularity statistics. These signals are compact, widely available, and still informative enough to reflect meaningful differences in session behavior. We organize them into four *cue families* in Section 3.2.

Idea 2: Coarse-to-Fine Multi-Scope Routing. RouteRec applies routing at three temporal scopes: macro, mid, and micro. Each scope uses a different observation range and captures a different kind of behavioral evidence, from broader cross-session tendency to immediate local transition. This gives the model a coarse-to-fine routing structure. Section 3.3 explains how each scope is defined and where it is applied in the backbone.

Idea 3: Hierarchical Sparse Expert Allocation. RouteRec organizes experts by cue family and applies sparse top- k routing in a hierarchical way. This makes expert selection more structured than using a single flat expert pool and helps different cue patterns lead to different computation paths. Section 3.4 describes this routing structure in detail.

Overall architecture. RouteRec is built on top of SASRec [16]. We keep the self-attention layers of SASRec and replace the original feed-forward transformations with routing-guided expert blocks. For layer ℓ , self-attention first produces contextualized hidden states

$$\tilde{\mathbf{H}}^{(\ell)} = \text{AttnBlock}(\mathbf{H}^{(\ell-1)}).$$

Given routing scope s , the router computes expert weights from the corresponding cue vector:

$$\boldsymbol{\pi}^{(s)} = \text{Router}^{(s)}(\mathbf{f}^{(s)}).$$

The selected experts then transform the hidden states:

$$\mathbf{H}^{(\ell)} = \sum_{e=1}^E \pi_e^{(s)} \text{Expert}_e^{(s)}(\tilde{\mathbf{H}}^{(\ell)}).$$

Figure 2 summarizes the overall framework of RouteRec.

Table 1: Representative cue families across routing scopes.

Family	Macro examples	Mid examples	Micro examples
Tempo	session gaps, pace trend	interval mean/std	last gap, local pace shift
Focus	theme consistency	category concentration, switching	local switch, suffix focus
Memory	repetition, carryover	repeat rate, novelty	recent reuse, longest run
Exposure	popularity tendency	popularity level, drift	last-item popularity, local spread

3.2 Behavioral Cue Construction

RouteRec builds routing cues from four cue families: *Tempo*, *Focus*, *Memory*, and *Exposure*. Each family captures a different aspect of session behavior. Tempo captures pace and interval patterns. Focus captures concentration and switching. Memory captures repetition and carryover. Exposure captures popularity level and drift. These families are built from common signals in user interaction history. Tempo is derived from timestamps. Focus is derived from coarse grouping labels when available. Memory is derived from item identities and repetition patterns. Exposure is derived from item popularity statistics computed from the training set. In this way, the cue bank stays compact while still covering the main differences in how sessions behave. The same four-family structure is used at all routing scopes. What changes across scopes is not the family definition itself, but the interaction range over which each family is summarized. Table 1 gives a compact overview of the cue families across scopes. The full cue definitions are listed in Table 5 and Appendix A.

3.3 Coarse-to-Fine Multi-Scope Routing

The next question is where each cue family should be used. Different behavioral signals become useful at different time ranges. Some require longer history, while others are visible only near the current prediction point. RouteRec handles this by assigning each routing stage its own observation range and its own routing point in the backbone.

More concretely, each routing scope is defined by the kind of behavior it is meant to capture and by the interaction range used to measure that behavior. For each routing scope s , RouteRec summarizes the corresponding interaction window $R_t^{(s)}$ into a cue vector

$$\mathbf{f}_t^{(s)} = \mathcal{A}\left(\mathcal{R}_t^{(s)}\right).$$

These cue vectors are passed to the matching routing blocks in the backbone.

The three scopes play different roles. Macro uses longer-history evidence from earlier sessions. Mid captures how the current session is developing so far. Micro focuses on the short local context right before prediction. This gives RouteRec a coarse-to-fine routing structure, where broader behavioral tendencies are handled earlier and more local behavior is handled closer to prediction.

RouteRec interleaves these routing stages with self-attention blocks instead of placing all routing decisions at one point. In the main configuration,

$$\text{self-attn} \rightarrow \text{macro} \rightarrow \text{mid} \rightarrow \text{self-attn} \rightarrow \text{micro}.$$

Macro and mid are applied as sequence-level routing stages, since both depend on broader session context rather than position-specific local changes. Micro is applied at the position level, since

short-horizon behavior can differ from one prediction step to the next. This layout keeps each routing stage tied to the time range it is meant to model. Full formulations are in Appendix A.

3.4 Hierarchical Sparse Expert Allocation

A critical challenge is ensuring that variations in behavioral cues lead to distinctly specialized computation paths. If weights spread across all experts, a repeat-heavy session and a fast-exploratory session may activate nearly the same mixture. RouteRec addresses this through *route commitment*: the model concentrates activation on a small, behavior-dependent expert subset. To do so, it organizes the E experts into G groups of C experts each ($E = GC$), where each group is aligned with one cue family (Tempo, Focus, Memory, or Exposure). The router first selects which cue family drives computation, then selects specific experts within that family. Sparse top- k selection at both levels ensures that two inputs with different cues cannot converge to the same route through soft averaging.

Router formulation. Given scope-specific cue vector $\mathbf{f}_t^{(s)}$, the router produces a hierarchical routing distribution over expert groups and experts within each group:

$$\pi_{t,g,c}^{(s)} = p_{t,g}^{(s)} r_{t,g,c}^{(s)}.$$

Here, $p_{t,g}^{(s)}$ is the group-level routing weight, and $r_{t,g,c}^{(s)}$ is the within-group expert weight. In this way, RouteRec first decides which cue families matter for the current input and then selects experts inside those families.

Sparse selection. To make routing selective, RouteRec applies hard top- k selection at both steps. It keeps only a small set of expert groups and, within each selected group, only a small set of experts. All unselected groups and experts receive zero weight, and the remaining weights are renormalized. This hierarchical sparse routing makes it easier for different cue patterns to activate different expert subsets.

3.5 Training Objective

RouteRec is trained end-to-end with one prediction loss and two routing regularizers. Maintaining route commitment requires the router to stay cue-sensitive throughout training. The prediction loss alone cannot guarantee this: cue-driven routing can degenerate in two distinct ways that the prediction loss does not penalize.

Prediction loss. The main objective is standard next-item cross-entropy. Let $\mathbf{z} \in \mathbb{R}^{|I|}$ be the final logit vector over the item vocabulary, and let $y \in I$ be the target item:

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(z_y)}{\sum_{j \in I} \exp(z_j)}. \quad (1)$$

Table 2: Recommendation performance of 10 different methods. Datasets marked with [†] are sampled processed subsets. Each method reports the run with the best validation performance under the shared selection rule. Lower Avg. Rank is better.

Dataset	Metric	SASRec	GRU4Rec	TiSASRec	FEARec	DuoRec	BSARec	FAME	DIF-SR	FDSA	RouteRec
KuaiRec [†]	HR@10	0.3284	0.3061	0.3216	<u>0.3513</u>	0.3468	0.3359	0.3307	0.2333	0.3453	0.3669
	NDCG@10	0.3177	0.2589	0.2889	<u>0.3286</u>	0.3215	0.3209	0.3188	0.2229	0.3197	0.3467
	MRR@20	0.3150	0.2462	0.2802	<u>0.3227</u>	0.3153	0.3170	0.3160	0.2201	0.3133	0.3425
Foursquare	HR@10	0.3049	0.2180	0.2976	0.3054	<u>0.3195</u>	0.2383	0.2417	0.3032	0.3042	0.3215
	NDCG@10	0.1929	0.1476	0.1926	0.1925	0.1970	0.1609	0.1604	0.2008	<u>0.2014</u>	0.2042
	MRR@20	0.1624	0.1286	0.1635	0.1613	0.1631	0.1393	0.1378	<u>0.1723</u>	0.1726	0.1715
LastFM [†]	HR@10	0.3775	0.3109	0.3715	<u>0.3876</u>	0.3850	0.3475	0.3359	0.3593	0.3762	0.3965
	NDCG@10	0.3097	0.2676	0.3029	0.3187	0.3066	0.3044	0.2972	0.3169	<u>0.3247</u>	0.3307
	MRR@20	0.2897	0.2558	0.2831	0.2981	0.2833	0.2915	0.2861	0.3051	<u>0.3103</u>	0.3113
Retail Rocket	HR@10	0.5321	0.5048	0.5131	<u>0.5487</u>	0.5478	0.4655	0.4612	0.4322	0.5343	0.5608
	NDCG@10	0.3907	0.3674	0.3755	<u>0.3998</u>	0.3994	0.3957	0.3943	0.3863	0.3848	0.4148
	MRR@20	0.3507	0.3290	0.3368	0.3581	0.3576	0.3745	<u>0.3738</u>	0.3709	0.3427	0.3737
Beauty	HR@10	0.1324	0.0840	0.1235	<u>0.1404</u>	0.1347	0.0726	0.0497	0.1165	0.1318	0.1561
	NDCG@10	0.0800	0.0457	0.0831	0.0797	0.0793	0.0504	0.0303	0.0811	<u>0.0918</u>	0.0948
	MRR@20	0.0659	0.0361	0.0735	0.0634	0.0662	0.0450	0.0250	0.0716	0.0822	<u>0.0794</u>
ML-1M	HR@10	0.1423	0.1456	0.1448	0.1490	0.1555	0.1374	0.1533	0.0669	0.1606	<u>0.1568</u>
	NDCG@10	0.0717	0.0778	0.0756	0.0736	0.0810	0.0723	<u>0.0818</u>	0.0331	0.0833	0.0807
	MRR@20	0.0555	0.0621	0.0600	0.0559	0.0641	0.0570	<u>0.0651</u>	0.0256	0.0655	0.0631
Avg. Rank↓		6.06	8.67	6.61	4.22	4.17	6.61	6.72	6.72	3.56	1.67

Route consistency. Routing collapse is a common failure mode in MoE: the router ignores behavioral cues and assigns behaviorally different sessions to nearly the same route. Route consistency regularizes the routing distribution so that it stays aligned with cue similarity. Intuitively, sessions with similar behavioral cues should induce similar routing distributions. For each scope s and sample i , let

$$\tilde{\mathbf{f}}_i^{(s)} = \frac{1}{L_i} \sum_t \mathbf{f}_{i,t}^{(s)}$$

be the session-level average cue vector, and let

$$\tilde{\boldsymbol{\pi}}_i^{(s)} = \frac{1}{L_i} \sum_t \boldsymbol{\pi}_{i,t}^{(s)}$$

be the corresponding average routing distribution. For each sample in the mini-batch, we find its nearest neighbor in cue space at scope s . We denote by $\mathcal{P}_s$ the resulting set of nearest-neighbor pairs (i, j) for that scope, and penalize the divergence between their routing distributions:

$$\mathcal{L}_{\text{cons}} = \frac{1}{|\mathcal{T}|} \sum_{s \in \mathcal{T}} \frac{1}{|\mathcal{P}_s|} \sum_{(i,j) \in \mathcal{P}_s} \text{JSD}(\tilde{\boldsymbol{\pi}}_i^{(s)}, \tilde{\boldsymbol{\pi}}_j^{(s)}).$$

This encourages the router to produce routing distributions that reflect the structure of the cue space.

Routing logit stabilization. A secondary, more implicit challenge arises even when the router remains cue-sensitive: the pre-softmax logits can grow very large during training. When this happens, the softmax saturates and routing collapses to a near-hard assignment. Small cue differences then become invisible to the router, so the cue vector stops mattering in practice. We address this with z-loss [43],

which penalizes the scale of the pre-softmax expert logits $a_{i,t,g,c}^s$ to keep the routing distribution in a well-behaved range where cue differences can produce meaningfully different outputs:

$$\mathcal{L}_z = \frac{1}{|\mathcal{T}|} \sum_{s \in \mathcal{T}} \frac{1}{N_s} \sum_{(i,t) \in \mathcal{V}_s} \left(\log \sum_{g,c} \exp a_{i,t,g,c}^s \right)^2,$$

where $\mathcal{V}_s$ is the set of valid positions and $N_s = |\mathcal{V}_s|$.

Overall objective function. The final objective is given as:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_z \mathcal{L}_z.$$

We do not include a load-balancing term. Classical MoE training often penalizes unequal expert usage to prevent collapse onto a small subset. Our goal is the opposite: behaviorally distinct sessions should activate different, committed routes rather than distributing traffic evenly. If a mini-batch is dominated by one behavioral regime, such as repeat-heavy sessions, then expert usage should also concentrate on the Memory-aligned route rather than be artificially flattened. Route consistency and logit stabilization together are sufficient for stable, cue-aligned specialization.

4 Experiments

This section evaluates RouteRec in terms of overall recommendation quality and design validity. We first compare RouteRec with strong sequential recommendation baselines on six public datasets. We then examine whether our behavior-guided routing is more effective than hidden-state routing, and test whether the proposed multi-scope routing and hierarchical sparse expert allocation are

both necessary. Finally, we study portability and routing behavior through reduced-cue variants and routing-profile analysis.

4.1 Experimental Setup

Datasets. We evaluate RouteRec on six public datasets: KuaiRec [7], Foursquare [40], LastFM [3], Retail Rocket [30], Beauty [26], and ML-1M [9]. All datasets are preprocessed under a shared protocol: interactions are sorted chronologically, grouped into sessions by inactivity threshold, and filtered to remove short sessions and low-support items. The resulting sessions are split chronologically into 70%/15%/15% train/validation/test partitions, and the prediction task is next-item prediction for the last item of each held-out session. KuaiRec and LastFM are large; we use 20% and 3% random subsets respectively, and all baselines are evaluated on the same data. Full preprocessing details and dataset statistics are in Appendix B.

Metrics. We report Hit Rate ($HR@k$), Normalized Discounted Cumulative Gain ($NDCG@k$), and Mean Reciprocal Rank ($MRR@k$) at $k \in \{5, 10, 20\}$. All evaluations use the seen-target setting, where the candidate set consists of items observed during training. This is standard practice for ID-based sequential recommendation, which cannot score items outside the training vocabulary [19]. The primary reported metrics are $HR@10$, $NDCG@10$, and $MRR@20$.

Baselines. We compare RouteRec against nine baselines in three groups. *Conventional sequential models*, namely GRU4Rec [10], SASRec [16], and TiSASRec [21], cover recurrent and self-attention backbones and represent the standard shared-computation baseline. *Representation-enhanced models*, namely DuoRec [27], FEARec [5], and BSARec [31], improve over the standard backbone through contrastive regularization or frequency-aware encoding, but still apply the same transformation to every session. *Side-information and MoE models*, namely FDSA [42], DIF-SR [39], and FAME [22], incorporate external item features or expert decomposition; they are the closest comparison to RouteRec in that they go beyond a purely shared backbone. All methods use the same processed data, the same optimization family, matched early stopping, and the same model-selection rule. Tuning details are in Appendix B.

4.2 Overall Results

Table 2 reports the main next-item prediction results on all datasets. Overall, RouteRec achieves the best average rank across datasets, outperforming all baselines in the aggregate comparison. It is especially strong on datasets where behavioral differences across sessions are more likely to matter, while remaining competitive even when the margin over the best baseline is small.

The gains are not uniform across datasets. On some datasets, RouteRec produces clear improvements over the strongest baseline, whereas on others the advantage is smaller or disappears. This is expected: RouteRec does not assume that all datasets require the same degree of routed computation. Instead, it is designed to help when session behavior differs enough that different computation paths become useful.

Even when the absolute gain is modest, the overall pattern is consistent. RouteRec remains highly competitive across all six datasets

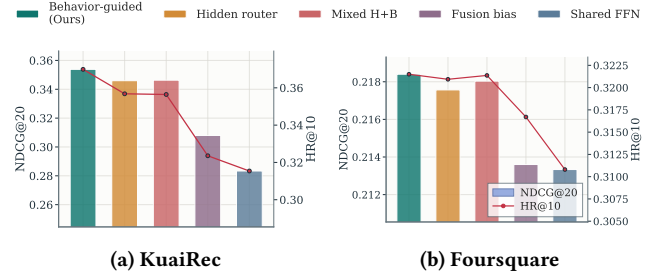


Figure 3: Routing-input ablations on KuaiRec and Foursquare. The five settings test whether the gain comes from hidden-state routing, behavioral cues, or an explicitly separated routing control path.

and achieves the best aggregate ranking, which suggests that guiding expert selection with behavioral cues is a robust alternative to shared-path modeling.

4.3 Routing Control

The overall comparison shows that RouteRec performs best on average, but does not directly isolate what makes the routing signal effective. The next question is whether behavioral cues are actually a better routing signal than hidden states of the backbone. Hidden states are optimized for next-item prediction, not for reflecting behavioral character, and routing from them ties the control path to the very representation it is meant to influence.

Figure 3 compares five routing-input settings: (1) the shared FFN baseline without routing, (2) hidden-only routing, where expert selection is driven only by backbone hidden states, (3) hidden+cues routing, where hidden states and behavioral cues are combined as router input, (4) behavior-guided routing, where expert selection is driven only by behavioral cues on a separate control path, and (5) a variant that injects cues into the backbone instead of keeping them separate.

The results show three clear patterns. First, hidden-only routing already improves over the shared FFN baseline, which suggests that routed computation itself is useful. Second, combining hidden states with behavioral cues can provide additional gains, but the improvement is not the most stable across datasets. Third, the behavior-guided router with a separate control path gives the most consistent results on both KuaiRec and Foursquare. This suggests that the main benefit comes not only from adding more information to the router, but from separating sequence modeling from expert selection so that routing can follow behavioral cues more directly.

4.4 Temporal Scope and Routing Design

The previous ablations show that behavioral cues are more effective than hidden states as routing signals. We next ask whether the routing structure itself is also necessary. RouteRec makes two design choices beyond the cue source: it separates routing into macro, mid, and micro scopes, and it organizes expert selection with hierarchical sparse routing. The first choice is meant to match different behavioral signals to the time range where they are most

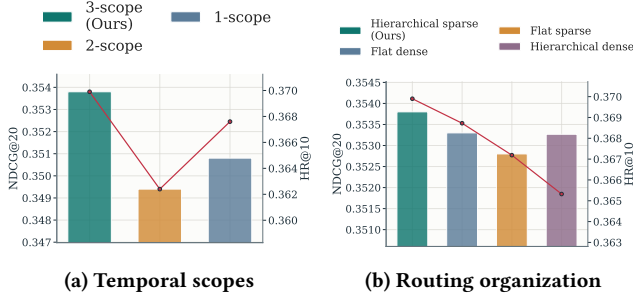


Figure 4: Routing-design ablations on KuaiRec. Panel (a) compares the full three-scope design with variants that remove one routing scope at a time. Panel (b) compares the proposed hierarchical sparse routing with flatter or denser alternatives under a comparable expert budget.

informative, while the second is meant to produce clearer and more selective expert choices. We test both design choices on KuaiRec.

Fig. 4a examines the temporal routing layout. The comparison starts from the full RouteRec design and removes one routing scope at a time, so the ablation tests whether each scope makes a distinct contribution. Fig. 4b examines routing organization. Here, the proposed hierarchical sparse routing is compared with flatter or denser routing variants under a similar expert budget, so the comparison isolates the effect of expert grouping and sparse selection. The results support both design choices. In Fig. 4a, the full three-scope layout performs best, and removing any one scope lowers performance. The largest drop appears when the macro scope is removed, which suggests that longer-range cross-session signals are not well replaced by within-session evidence alone. In Fig. 4b, both flat routing and dense activation perform worse than the hierarchical sparse design. This indicates that expert grouping and sparse route selection each help improve routing quality.

4.5 Cue Portability

Cue portability matters because item IDs, timestamps, and category labels are not always all available in practice. We therefore retrain RouteRec from scratch under three reduced-cue conditions: removing category cues, removing timestamp cues, or removing both. We compare these variants against the full model and the best baseline.

Figure 5 compares five settings on each dataset. The first bar is the best non-RouteRec baseline. The second is the full RouteRec model. The remaining three bars are reduced-cue variants retrained from scratch after removing category cues, timestamp cues, or both. This experiment tests whether RouteRec depends too strongly on any one commonly available cue source.

The results show that RouteRec remains effective even when some cue sources are missing. The full model performs best overall, but the reduced-cue variants remain generally competitive with the best baseline, and on one dataset all RouteRec variants still outperform that baseline. This suggests that category and timestamp cues are useful, but neither is strictly required on its own for behavior-guided routing to remain effective. In other words, RouteRec benefits from multiple common cue sources, without relying entirely on any single one of them.

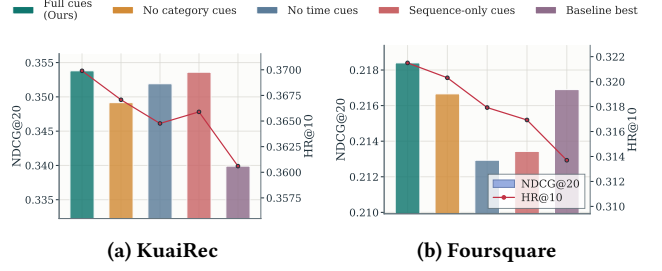


Figure 5: Cue-portability results on KuaiRec and Foursquare. The bars compare the full RouteRec model, reduced-cue variants trained without category cues, timestamp cues, or both, and the best baseline. This experiment tests whether behavior-guided routing remains effective when some commonly available cue sources are missing.

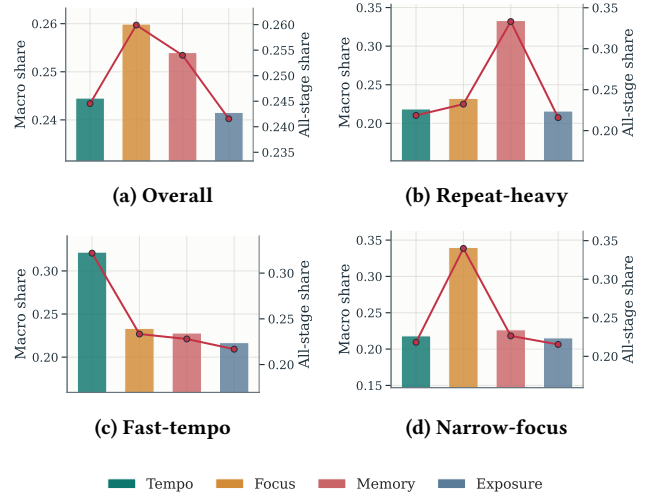


Figure 6: Family-weight routing profiles for four user subsets on KuaiRec: (a) the full test population as an anchor and (b–d) subsets selected for elevated repeat intensity, fast interaction pace, and narrow topical focus.

4.6 Routing Profiles

The preceding ablations show that the design components are necessary. A complementary question is whether the router actually responds to behavioral differences between sessions. If expert allocation is behavior-guided, sessions with different behaviors should produce different routing at each routing scope.

Figure 6 shows the average family-level routing weights for several user subsets on KuaiRec. The subsets are constructed to isolate one behavioral pattern at a time. For each target cue, we rank users by the corresponding user-level cue score and select the top 10%. We then collect the sessions of those users and compute the average routing weights over cue families. Under behavior-guided routing, a subset chosen for one dominant cue should place more routing weight on the corresponding expert family.

The results match this expectation. The repeat-heavy subset places more weight on the Memory family, especially at the macro scope where cross-session repetition is most visible. The fast-tempo subset shows higher Tempo weight, most clearly at the mid and micro scopes where pace and interval changes are freshest. The narrow-focus subset places more weight on the Focus family, especially in the within-session scopes where concentration is easiest to observe. Overall, these shifts show that the router responds to the targeted behavioral cues and assigns different family-level routing patterns to behaviorally different user groups.

5 Conclusion

In this work, we propose RouteRec, a sequential recommendation model that uses behavioral cues to guide expert routing. The key idea is that different sessions can behave differently, so they should not always follow the same computation path. RouteRec captures these differences from common signals in user interaction history and uses them on a separate routing path.

RouteRec combines three ideas: behavioral cue construction, coarse-to-fine multi-scope routing, and hierarchical sparse expert allocation. Together, these components let the model use both broader session tendencies and local next-step behavior when selecting experts, while keeping routing separate from the main sequence backbone.

Across six public datasets, RouteRec achieves the best overall average rank. The gains are especially clear when the data shows stronger behavioral differences across sessions and enough earlier-session context is available. More broadly, the results suggest that guiding expert selection with behavioral cues is a useful alternative to shared-path modeling in sequential recommendation.

There are several directions for future work. One is to study how behavior-guided routing transfers across datasets with different domains or metadata conditions. Another is to extend the cue design beyond the current set of sequence-derived signals. We also expect that the same routing idea could be applied to other recommendation backbones beyond SASRec.

References

- [1] Shuqing Bian, Xingyu Pan, Wayne Xin Zhao, Jinpeng Wang, Chao Wang, and Ji-Rong Wen. 2023. Multi-modal Mixture of Experts Representation Learning for Sequential Recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*. 110–119. doi:10.1145/3583780.3614978
- [2] Simone Borg Bruun, Maria Maistro, and Christina Lioma. 2022. Learning Recommendations from User Actions in the Item-Poor Insurance Domain. In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*. Association for Computing Machinery, New York, NY, USA, 113–123. doi:10.1145/3523227.3546775
- [3] Oscar Celma. 2010. *Music Recommendation and Discovery in the Long Tail*. Springer. doi:10.1007/978-3-642-13287-2
- [4] Jianxin Chang, Chenbin Zhang, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. PEPNet: Parameter and Embedding Personalized Network for Infusing with Personalized Prior Information. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 379–389. doi:10.1145/3580305.3599884
- [5] Xinyu Du, Huanhuan Yuan, Pengpeng Zhao, Jianfeng Qu, Fuzhen Zhuang, Guafeng Liu, Yanchi Liu, and Victor S. Sheng. 2023. Frequency Enhanced Hybrid Attention Network for Sequential Recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 78–88. doi:10.1145/3539618.3591689
- [6] B. G. Galuzzi, I. Giordani, A. Candelieri, R. Perego, and F. Archetti. 2020. Hyperparameter Optimization for Recommender Systems Through Bayesian Optimization. *Computational Management Science* 17, 4 (2020), 495–515. doi:10.1007/s10287-020-00376-3
- [7] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A Fully-Observed Dataset and Insights for Evaluating Recommender Systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (Atlanta, GA, USA) (CIKM '22)*. 540–550. doi:10.1145/3511808.3557220
- [8] Dmytro Gusak, Aleksandra Mojsilovic, Vassilis N. Ioannidis, and Christos Kozyrakis. 2025. Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders. arXiv:2507.16289 [cs.LR] doi:10.48550/arXiv.2507.16289
- [9] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 1–19. doi:10.1145/2827872
- [10] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1511.06939>
- [11] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive Mixtures of Local Experts. *Neural Computation* 3, 1 (1991), 79–87. doi:10.1162/neco.1991.3.1.79
- [12] Yash Jain, Harkirat Behl, Zsolt Kira, and Vibhav Vineet. 2023. DAMEX: Dataset-aware Mixture-of-Experts for Visual Understanding of Mixture-of-Datasets. In *Advances in Neural Information Processing Systems*.
- [13] Dietmar Jannach, Massimo Quadrana, and Paolo Cremonesi. 2022. Session-Based Recommender Systems. In *Recommender Systems Handbook*. Springer. doi:10.1007/978-1-0716-2197-4_8
- [14] Yuyi Ji, Aline B. de Oliveira, Raphael D. F. Mesquita, Diego H. C. G. Horta, and Leonardo B. Marinho. 2023. A Critical Study on Data Leakage in Recommender System Offline Evaluation. *ACM Transactions on Information Systems* 41, 3 (2023), 1–27. doi:10.1145/3569930
- [15] Michael I. Jordan and Robert A. Jacobs. 1994. Hierarchical Mixtures of Experts and the EM Algorithm. *Neural Computation* 6, 2 (1994), 181–214. doi:10.1162/neco.1994.6.2.181
- [16] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. doi:10.1109/ICDM.2018.00035
- [17] Emanuel Lacic, Markus Reiter-Haas, Dominik Kowald, Manoj Reddy Daredy, Junghoo Cho, and Elisabeth Lex. 2020. Using Autoencoders for Session-Based Job Recommendations. *User Modeling and User-Adapted Interaction* 30, 4 (2020), 617–658. doi:10.1007/s11257-020-09269-1
- [18] Sara Latifi, Dietmar Jannach, and Andrés Ferraro. 2022. Sequential Recommendation: A Study on Transformers, Nearest Neighbors and Sampled Metrics. *Information Sciences* 609 (2022), 660–678. doi:10.1016/j.ins.2022.07.079
- [19] Sahar Latifi, Nicola Mauro, and Dietmar Jannach. 2021. Session-Aware Recommendation: A Surprising Quest for the State-of-the-Art. *Information Sciences* 573 (2021), 354–375. doi:10.1016/j.ins.2021.05.048
- [20] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural Attentive Session-based Recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*. 1419–1428. doi:10.1145/3132847.3132926
- [21] Jiacheng Li, Yujie Wang, and Julian J. McAuley. 2020. Time Interval Aware Self-Attention for Sequential Recommendation. In *Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM)*. 322–330. doi:10.1145/3336191.3371786
- [22] Mingrui Liu, Sixiao Zhang, and Cheng Long. 2025. Facet-Aware Multi-Head Mixture-of-Experts Model for Sequential Recommendation. In *Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM)*. 127–135. doi:10.1145/3701551.3703552
- [23] Qiao Liu, Yifu Zeng, Refuoe Mokhosi, and Haibin Zhang. 2018. STAMP: Short-Term Attention/Memory Priority Model for Session-Based Recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1831–1839. doi:10.1145/3219819.3219950
- [24] Ziwei Liu, Qidong Liu, Yejing Wang, Wanyu Wang, Pengyue Jia, Maolin Wang, Zitao Liu, Yi Chang, and Xiangyu Zhao. 2025. SIGMA: Selective Gated Mamba for Sequential Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 12 (2025), 12264–12272. doi:10.1609/aaai.v39i12.33336
- [25] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi. 2018. Modeling Task Relationships in Multi-Task Learning with Multi-Gate Mixture-of-Experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1930–1939. doi:10.1145/3219819.3220007
- [26] Julian McAuley, Rahul Pandey, and Jure Leskovec. 2015. Inferring Networks of Substitutable and Complementary Products. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 785–794. doi:10.1145/2783258.2783381
- [27] Ruihong Qiu, Zi Huang, Hongzhi Yin, and Zijian Wang. 2022. Contrastive Learning for Representation Degeneration Problem in Sequential Recommendation. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*. 813–823. doi:10.1145/3488560.3498433

- [28] Massimo Quadran, Alexandros Karatzoglou, Balázs Hidasi, and Paolo Cremonesi. 2017. Personalizing Session-based Recommendations with Hierarchical Recurrent Neural Networks. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (RecSys)*. 130–137. doi:10.1145/3109859.3109896
- [29] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing Personalized Markov Chains for Next-Basket Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*. 811–820. doi:10.1145/1772690.1772773
- [30] Retail Rocket. 2015. Retail Rocket Recommender System Dataset. <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>
- [31] Yehjin Shin, Jeongwhan Choi, Hyowon Wi, and Noseong Park. 2024. An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 8 (2024), 8984–8992. doi:10.1609/aaai.v38i8.28747
- [32] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450. doi:10.1145/3357384.3357895
- [33] Hongyan Tang, Junling Liu, Ming Zhao, Xudong Gong, Zhaohui Jin, Yu Lei, Min Zhang, and Xuan Li. 2020. Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys)*. 269–278. doi:10.1145/3383313.3412236
- [34] Jiaxi Tang and Ke Wang. 2018. Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding. In *Proceedings of the 11th ACM International Conference on Web Search and Data Mining (WSDM)*. 565–573. doi:10.1145/3159652.3159656
- [35] Shoujin Wang, Longbing Cao, Yan Wang, Quan Z. Sheng, Mehmet A. Orgun, and Defu Lian. 2021. A Survey on Session-Based Recommender Systems: Challenges, Progress and Prospects. *Comput. Surveys* 54, 7 (2021), 1–38. doi:10.1145/3465401
- [36] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z. Sheng, and Mehmet A. Orgun. 2019. Sequential Recommender Systems: Challenges, Progress and Prospects. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 6332–6338. doi:10.24963/ijcai.2019/883
- [37] Wenhao Wang, Zhaojie Luo, Zhuo Chen, Jidong Chen, Ming Lei, and Lei Xie. 2023. Language-Routing Mixture of Experts for Multilingual and Code-Switching Automatic Speech Recognition. In *Interspeech 2023*. doi:10.21437/Interspeech.2023-2292
- [38] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-Based Recommendation with Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 1 (2019), 346–353. doi:10.1609/aaai.v33i01.3301346
- [39] Yueqi Xie, Peilin Zhou, and Sunghun Kim. 2022. Decoupled Side Information Fusion for Sequential Recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 1611–1621. doi:10.1145/3477495.3531963
- [40] Dingqi Yang, Daqing Zhang, Vincent W. Zheng, and Zhiyong Yu. 2015. Modeling User Activity Preference by Leveraging User Spatial Temporal Characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45, 1 (2015), 129–142. doi:10.1109/TSMC.2014.2327053
- [41] Shengzhe Zhang, Liyi Chen, Dazhong Shen, Chao Wang, and Hui Xiong. 2025. Hierarchical Time-Aware Mixture of Experts for Multi-Modal Sequential Recommendation. In *Proceedings of the ACM Web Conference 2025 (WWW)*. 3672–3682. doi:10.1145/3696410.3714676
- [42] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfang Liu, and Xiaofang Zhou. 2019. Feature-level Deeper Self-Attention Network for Sequential Recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 4320–4326. doi:10.24963/ijcai.2019/600

- [43] Barret Zoph, Alexander V. Kolesnikov, Neal Wadhwa, Jonathan Huang, Andrew M. Dai, Noam Shazeer, Quoc V. Le, and Yonghui Wu. 2022. Designing Effective Sparse Expert Models. arXiv:2202.08906 [cs.LG] doi:10.48550/arXiv.2202.08906

A Exact RouteRec Instantiation and Sparse Routing Formulation

This appendix records the fixed configuration and the representative cue bank used by RouteRec. The main text emphasizes the conceptual design; this section documents the concrete implementation choices needed for reproducibility and for interpreting the later ablations, including the full sparse routing formulation.

A.1 Main Paper Configuration

Table 3: Fixed main configuration used in the paper.

Aspect	Main paper configuration
Backbone layout	self-attn → macro block → mid block → self-attn → micro block
Scope execution	Serial
Routing granularity	Macro and mid: sequence-wise; micro: position-wise
Router composition	Shared hierarchical sparse router across all three scopes
Expert activation	Top-3 groups out of 4; top-2 experts per selected group
Active experts	6 out of 12 total per routed decision
History windows	$H = 5$ sessions (macro); $w = 5$ positions (micro)
Core auxiliary terms	Route consistency + z-loss; no load balancing

The placement of the two attention layers reflects the intended temporal roles of the scopes. The first attention layer contextualizes the raw sequence before any routing is applied. Macro and mid then operate consecutively because both provide slower sequence-level control over the same contextualized sequence state: macro captures broader recent-history tendency, while mid refines within-session evolution. A second attention layer is inserted only before micro so that the final local router receives a refreshed token interaction state immediately before prediction. This layout was the most stable high-performing configuration in our architecture sweep, and Appendix D compares it against denser and differently ordered alternatives.

This table serves as a reproducibility summary rather than as a search-space description. Architectural choices are fixed across the paper track, whereas capacity and optimization values are selected per dataset from a narrow candidate set and are therefore reported as compact ranges only when no single global value is shared across all reported runs.

A.2 Implementation Specification and Minimal Execution Notes

Table 4: Implementation-level settings used in the selected paper track.

Item	Reported setting
Semantic groups	$G = 4$ groups: Tempo, Focus, Memory, Exposure
Experts per group	$C \in \{2, 3, 4\}$; total experts $E = 4C$
Main sparse setting	$C = 3$, top-3 groups, top-2 experts per selected group
Cue dimensionality	16 scalars per scope
Router hidden width	Representative range 48–128
Routing temperature	Macro 1.0; mid and micro 1.2
Consistency neighbors	$k_{nn} = 1$ nearest non-self sample per scope
Missing-signal handling	Broadcast when available; zero-fill focus cues if grouping field absent

Minimal execution notes. The implementation uses three simple rules that are easy to miss if one only reads the high-level method description. First, macro and mid cues are computed once per example and then broadcast over the routed positions, whereas micro cues are evaluated at the position level. Second, top- k masking is followed by re-normalization inside the active support rather than by leaving the logits in an unnormalized sparse state. Third, if a dataset does not expose the grouping field needed for focus-family cues, the implementation falls back to the fixed missing-signal policy in Table 4 rather than silently redefining the cue family per dataset.

A.3 Representative Behavioral Cue Families

This cue bank is intentionally compact. The cue families are hand-designed in the sense that they encode behavioral hypotheses, but the design goal is to keep them as a small set of summaries that can serve as a stable routing interface across datasets. In practice, the macro bank is instantiated from the recent-session window used by the selected paper instantiation, while mid and micro are computed from the current session context at different temporal scopes. The main text therefore reports the family structure and representative examples, while this appendix provides the implementation-oriented view.

B Dataset, Evaluation, and Full Ranking Results

All experiments were conducted on machines equipped with NVIDIA RTX 3090 24GB GPUs. Each training or evaluation process used a single GPU; multiple GPUs were used only to run independent experiments in parallel, not for multi-GPU training within a single run.

This appendix summarizes the six datasets and makes the evaluation protocol explicit, since recent work has shown that session construction and split strategy can materially affect offline recommender results [8, 14]. The original releases are cited here for completeness [3, 7, 9, 26, 30, 40]. The dataset descriptions emphasize domain differences and signal availability because the routing branch depends on which behavioral evidence is exposed in each corpus.

Table 5: Representative cue summaries used by RouteRec in the main configuration. Each scope uses four cue families with four scalar summaries per family; the table lists representative examples rather than every implementation detail.

Family	Macro (recent-session tendency)	Mid (within-session evolution)	Micro (short-horizon pressure)
Tempo	Context-valid ratio, last inter-session gap, mean pace, pace trend	Valid-context ratio, mean/std interaction interval, session age	Last gap, short-window mean gap, short-window std, deviation from mid-scale pace
Focus	Theme entropy, dominant-theme mass, theme repeat rate, theme shift rate	Category entropy, category top-1 mass, category switch rate, category uniqueness	Immediate category switch, mismatch to recent category history, suffix entropy, suffix uniqueness
Memory	Repeat intensity, adjacent category overlap, adjacent item overlap, repeat trend	Item uniqueness, repeat rate, novelty rate, longest item run	Reconsumption flag, suffix reconsumption rate, suffix item uniqueness, suffix longest run
Exposure	Mean popularity, popularity spread, popularity entropy, popularity trend	Mean popularity, popularity std, popularity entropy, popularity drift	Last-item popularity, suffix popularity spread, suffix popularity entropy, deviation from mid-scale popularity

B.1 Datasets and Protocol

Table 6: Datasets used in the experiments. Datasets marked with [†] are sampled processed subsets.

Dataset	Domain	Interactions	Sessions	Items	Avg. len.
KuaiRec [†]	Video	287,411	24,458	6,477	11.8
Foursquare	POI	145,238	25,369	30,588	5.7
LastFM [†]	Music	470,408	25,089	52,510	18.8
Retail Rocket	Browse	821,243	153,092	90,211	5.4
Beauty	E-com.	33,488	4,243	3,625	7.9
ML-1M	Movie	575,281	14,539	3,533	39.6

Table 6 reports the statistics of the final processed datasets used in the experiments rather than the raw corpus sizes. In particular, KuaiRec corresponds to the 20% processed subset used in our runs, and LastFM corresponds to the 3% processed subset.

For reproducibility, the protocol is summarized as follows.

- (1) Sort all interactions chronologically within each dataset.
- (2) Reconstruct sessions with a dataset-specific inactivity threshold.
- (3) Iteratively prune sessions with length below 5 and items with support below 3 until the constraints are simultaneously satisfied.
- (4) Sort the resulting sessions by session start time.
- (5) Assign the earliest 70% of sessions to train and treat the latest 30% as a held-out tail.
- (6) Build validation and test from the held-out tail under chronology-preserving cleanup.

KuaiRec and Foursquare emphasize stronger short-horizon intent changes, LastFM provides long repeated listening histories, Retail Rocket and Beauty stress sparse revisit behavior, and ML-1M provides longer preference-driven sequences. This spread makes it possible to examine whether behavior-guided routing is primarily beneficial in highly dynamic settings or remains competitive when preference structure is more stable.

Signal availability also differs in a way that matters for routing. KuaiRec offers item, time, and coarse content-side metadata in short-video sessions with strong short-term intent shifts. Foursquare includes item, timestamp, and venue-category structure with bursty check-in dynamics. LastFM provides repeated listening histories with artist-side signals. Retail Rocket is the sparsest side-information setting, relying mainly on anonymous event streams with item and time signals. Beauty exposes item, time, and product-category cues after low-rating removal, while

ML-1M contributes longer preference-driven sequences with genre information.

We position our setup as a sessionized sequential evaluation protocol assembled from established components, not as the canonical raw-sequence protocol for sequential recommendation [13, 19, 28]. Sessions are reconstructed post hoc with inactivity heuristics, which is standard practice when logs do not provide explicit session boundaries [13]. The thresholds are adapted to the logging regime of each corpus: Foursquare, KuaiRec, LastFM, ML-1M, and Retail Rocket use a 30-minute inactivity threshold, consistent with dense-log session-aware practice [17, 28], whereas Beauty uses a 14-day threshold because review timestamps are substantially more diffuse. More generally, this threshold choice reflects the observation that sparse and item-poor domains often require longer or more domain-specific notions of a session than dense clickstream data [2]. After sessionization, we repeatedly remove sessions shorter than 5 and items with fewer than 3 interactions until no further pruning is required. This iterative filtering follows the same general rationale as common session-based benchmark preprocessing, where short sessions and very low-support items are filtered to stabilize next-item supervision and evaluation [20, 38].

Session splits are constructed at the session level after sorting by session start time. The earliest 70% of sessions are assigned to training, and the remaining 30% are treated as a held-out tail. This tail is first divided into validation and test using stratification over three factors: whether the target item already appears in the history, the target item’s popularity bucket with respect to the training split, and the session-length bucket. For the final evaluation files, we then merge the held-out sessions, restore chronological order, and re-split them contiguously into equal validation and test halves. This chronology-preserving design is meant to reduce leakage and keep the offline protocol aligned with temporal deployment order [8, 14].

For the primary evaluation view, we report seen-target metrics only. Concretely, in the held-out files, non-target rows whose items are unseen in training are removed, and unseen targets are retained only for supplementary accounting rather than the main comparison. This matches long-standing session-based and session-aware evaluation practice for ID-based recommendation models, which cannot rank targets outside the training vocabulary and therefore typically separate unseen-target behavior from the primary benchmark table [19, 20, 38]. We report HR, NDCG, and MRR at $k \in \{5, 10, 20\}$ for all methods.

For the result tables, each method is represented by the run with the best mean validation performance over the nine reported metrics under the same selection rule used throughout the paper.

All methods use the same processed train/validation/test partitions. RouteRec’s additional macro, mid, and micro cues are deterministic summaries derived from those same observed histories, so the comparison should be read as evaluating different ways of exploiting the shared sessionized interaction context rather than different access to held-out information.

B.2 Model Selection and Tuning Protocol

Because this benchmark is built from a sessionized temporal protocol rather than a canonical raw-sequence leaderboard, the model-selection procedure is stated explicitly. Prior benchmark studies have shown that both baseline quality and model ranking can change materially once methods are re-evaluated under a common framework and retuned under a shared protocol [18, 19]. Accordingly, the comparison does not rely on paper defaults alone.

Our reporting protocol follows three simple rules. First, every model is tuned within a predefined bounded search space that is fixed before the final comparison: learning rate is sampled log-uniformly within a declared dataset-specific interval, while the remaining hyperparameters are drawn from small predeclared candidate sets. Second, these bounded spaces cover a shared core of optimization hyperparameters together with a small number of model-specific controls, so that the search is broad enough to avoid trivial under-tuning but still budgeted tightly enough to remain comparable across methods. Third, the final reported result for every model is the test performance of the configuration that maximizes the mean validation score over the nine reported metrics. When adaptive search is used for computational efficiency, it is restricted to the same predeclared bounds and candidate sets and should therefore be read as an efficient implementation of a controlled bounded search rather than as open-ended retuning [6].

The final paper track therefore uses bounded search spaces rather than unconstrained retuning. In the paper-facing comparison, baselines and RouteRec are aligned to the same shared optimization pool, and model families differ only through a small number of architecture-specific additions. For RouteRec, the selected three-scope routing layout remains fixed, but depth, hidden dropout, and routing-side controls are still tuned within bounded candidate sets. To keep the appendix concise but still reproducible, Table 7 separates the shared optimization pool from the baseline-family and RouteRec-specific additions, and lists the dataset-specific learning-rate intervals explicitly.

B.3 Full Ranking Results

Table 8 restores the full cutoff grid summarized more compactly in Table 2. This presentation preserves the same selection rule as the main text while exposing the complete $k \in \{5, 10, 20\}$ breakdown.

The full table also helps interpret the average-rank summary in the main text. RouteRec stays competitive across all cutoffs instead of relying on a single metric or a single rank threshold, and the strongest competing baseline still varies by dataset. This is why we emphasize the overall pattern rather than any one pairwise comparison.

C Dataset Analysis and Behavioral Indicators

This section defines the four dataset-level indicators used in the main text and reports the full corpus profile behind them. All indicators are computed from the training split before any model training. They are descriptive summaries of recurring behavioral differences in each dataset, not model-specific scores.

C.1 Indicator Definitions

Each indicator is a dataset-level scalar derived from the observed training data. All use scale-free (ratio or normalized) forms so that datasets with different interaction volumes remain comparable.

- **Temporal volatility** measures how variable session length is within the corpus. It is the coefficient of variation of session lengths across all sessions:

$$\text{SessVol} = \frac{\text{std}(\text{session_len})}{\text{mean}(\text{session_len})}.$$

A higher value means session pace and duration are more irregular, which raises the case for tempo-sensitive routing.

- **Transition branching** measures how ambiguous local next-item transitions are. For each source item i , let $P_i(j)$ be the empirical probability of transitioning from i to item j , and let $|\text{supp}(P_i)|$ be the number of distinct observed next items. The normalized next-item entropy for item i is

$$H_{\text{norm}}(i) = \frac{-\sum_j P_i(j) \log P_i(j)}{\log |\text{supp}(P_i)|} \in [0, 1].$$

The dataset-level score is the transition-count-weighted average over all items with at least two distinct next items:

$$\text{Branch} = \frac{\sum_i n_i H_{\text{norm}}(i)}{\sum_i n_i},$$

where n_i is the number of observed transitions from item i . A higher value means the same local state tends to branch into many different next items, making next-step intent more ambiguous.

- **Memory variability** measures how unevenly repetition behavior is distributed across sessions. For each session, the repeat ratio is $\text{repeat_ratio} = 1 - (\text{unique items}/\text{session length})$. The dataset-level score is the coefficient of variation of this ratio across all sessions:

$$\text{RepeatVar} = \frac{\text{std}(\text{repeat_ratio})}{\text{mean}(\text{repeat_ratio})}.$$

A value of zero means all sessions repeat at the same rate. A higher value means some sessions are strongly repeat-heavy while others are not, making repetition a more variable and routing-relevant axis.

- **Context availability** measures how much cross-session history is available for macro-scope routing. It is a composite of three sub-indicators—the fraction of users with at least 2 sessions ($r_{\geq 2}$), the fraction with at least 5 sessions ($r_{\geq 5}$), and a clipped normalized session count $\min(\bar{s}/20, 1)$ where $\bar{s}$ is the mean number of sessions per user:

$$\text{CtxAvail} = \frac{1}{3}(r_{\geq 2} + r_{\geq 5} + \min(\bar{s}/20, 1)).$$

A higher value means more users have enough earlier-session history for macro-scope routing to use. This is not itself a cue family; instead, it indicates whether broader cross-session evidence is available often enough to matter.

Table 7: Bounded candidate grids used in the final paper track.**(a) Shared Optimization Pool**

Parameter	Candidate sets
Learning rate	Dataset-specific log-uniform interval; see panel (d)
Weight decay	$\{5 \times 10^{-7}, 10^{-6}, 10^{-5}, 5 \times 10^{-5}, 10^{-4}, 1.5 \times 10^{-4}\}$
Max history length	$\{10, 20, 30\}$ or $\{10, 20, 30, 50\}$
Width / embedding	$\{64, 96, 112, 128, 160\}$ or $\{96, 112, 128, 160, 192\}$
Inner width	$\{128, 192, 224, 256, 320\}$ or $\{192, 224, 256, 320, 384\}$
Layers / heads	Layers $\{1, 2, 3, 4\}$; heads $\{1, 2, 4, 8\}$ if applicable
Dropout	Hidden / ratio: $\{0.10, 0.12, 0.13, 0.15, 0.18\}$ Recurrent: $\{0.10, 0.15, 0.20, 0.25, 0.30\}$ Attention: $\{0.06, 0.08, 0.10, 0.12, 0.15\}$ or $\{0.08, 0.10, 0.12, 0.15, 0.20\}$

(c) RouteRec-Specific Additions

Parameter	Candidate sets
Backbone depth / dropout	Depth: $\{1, 2, 3\}$ Dropout: $\{0.10, 0.12, 0.14, 0.15, 0.16, 0.17\}$ $\{0.18, 0.19, 0.20, 0.22, 0.24\}$
Weight decay	$\{3.125, 5, 6, 10, 15, 20, 50\} \times 10^{-7}$
Routing capacity	Expert scale $\{2, 3, 4\}$ or $\{2, 3, 4, 5\}$ for KuaiRec router width $\{32, 64, 96, 128\}$
Routing dropout	Feature $\{0, 0.03, 0.05, 0.10\}$ attention $\{0.05, 0.06, 0.08, 0.10, 0.12\}$
Regularization	$\lambda_{\text{cons}} \{0, 2.5, 5, 8, 12\} \times 10^{-4}$ $\lambda_z \{0, 5, 10, 20\} \times 10^{-5}$
Cue bank	Cue bank size $\{10, 12, 16, 24\}$ router inner width per cue family (small preset)

C.2 Full Dataset-Level Axis Profile

Table 9 extends the four indicators in the main text with three additional corpus summaries: Carryover (mean cross-session item overlap), Drift (how strongly consecutive sessions change), and PopConc (popularity concentration, measured as the Gini coefficient of item frequency).

Table 9: Full dataset-level axis profile across datasets. All values are computed from the training split before model training. Gain is RouteRec’s mean test-score gap from the strongest baseline; WinRate is the fraction of nine reported metrics where RouteRec ranks first.

Dataset	Temporal volatility	Transition branching	Memory variability	Context availability	Carryover	Drift	PopConc	Gain	WinRate
Beauty	0.694	0.843	0.000	0.059	0.000	1.000	0.509	0.0020	0.556
Foursquare	0.944	0.688	1.961	0.988	0.159	0.841	0.647	0.0009	0.556
KuaiRec	0.887	0.903	2.214	0.933	0.003	0.997	0.671	0.0182	1.000
LastFM	0.739	0.630	2.036	0.990	0.072	0.928	0.456	0.0049	1.000
ML-1M	0.381	0.839	0.000	0.264	0.000	1.000	0.699	-0.0031	0.000
Retail Rocket	0.945	0.744	0.858	0.056	0.184	0.816	0.674	0.0055	0.667

The four indicators used in the main text cover distinct recurring behavioral axes: temporal structure (volatility), local transition structure (branching), repetition structure (memory variability), and the availability of earlier-session evidence (context availability). The three remaining columns extend this view with cross-session continuity and popularity structure.

C.3 Supplementary Column Definitions

- **Carryover** measures how much item content carries over between consecutive sessions. For each user, let I_m be the item set of session s_m . The Jaccard overlap between adjacent sessions is $|I_{m-1} \cap I_m| / |I_{m-1} \cup I_m|$, and the dataset-level value is the mean over all consecutive session pairs $\mathcal{P}$:

$$\text{Carryover} = \frac{1}{|\mathcal{P}|} \sum_{(m-1, m) \in \mathcal{P}} \frac{|I_{m-1} \cap I_m|}{|I_{m-1} \cup I_m|}.$$

(b) Baseline-Family Additions

Model(s)	Additional candidate sets
TISASRec	Time span $\{64, 128, 256, 384, 512\}$
GRU4Rec	Recurrent dropout $\{0.10, 0.15, 0.20, 0.25, 0.30\}$
DuoRec / FEARec	$\tau \{0.16, 0.18, 0.20, 0.22, 0.24\}$ Contrastive weight $\{0.02, 0.03, 0.04, 0.05, 0.06\}$ Semantic weight $\{0, 0.04, 0.05, 0.08, 0.10\}$ or $\{0.04, 0.05, 0.08, 0.10, 0.12\}$
BSARec	$\alpha \{0.35, 0.50, 0.55, 0.70\}$ $c \{2, 3, 5, 7\}$
DIF-SR / FDSA	Attribute width $\{96, 128, 160, 192\}$ $\lambda_{\text{attr}} \{0.08, 0.09, 0.10, 0.12, 0.14\}$ or $\{0.09, 0.10, 0.12, 0.14, 0.15\}$
DIF-SR	Fusion $\{\text{gate, sum, concat}\}$
FAME	Expert count $\{2, 3, 4, 5, 6\}$

(d) Dataset-Specific Learning-Rate Intervals

Dataset	Unified log-uniform LR interval
KuaiRec	$3e-4 \sim 5e-3$
Foursquare	$1.5e-4 \sim 2.2e-3$
LastFM	$8e-5 \sim 1.2e-3$
Retail Rocket	$1.5e-4 \sim 2.2e-3$
Beauty	$1.5e-4 \sim 2e-3$
ML-1M	$1.5e-4 \sim 2.2e-3$

Near-zero values (Beauty, KuaiRec, ML-1M) indicate that consecutive sessions are almost entirely item-disjoint. Retail Rocket and Foursquare show moderate carryover, consistent with return visits to familiar locations or product categories.

- **Drift** is the complement of Carryover: $\text{Drift} = 1 - \text{Carryover}$. It reflects how strongly one session moves away from the previous one. A value near 1 means consecutive sessions rarely share items, so broader history is less about exact item reuse and more about slower behavioral tendency.
- **PopConc** (popularity concentration) measures how unevenly interactions are distributed across the item catalog. It is the Gini coefficient of the empirical item frequency distribution $\{f_i\}$:

$$\text{PopConc} = \frac{\sum_i \sum_j |f_i - f_j|}{2N \sum_i f_i},$$

where N is the number of items. Values range from 0 (uniform distribution) to 1 (all interactions on one item). ML-1M and Retail Rocket have the highest concentration (0.699 and 0.674), reflecting popular movie and product head effects. LastFM is the lowest (0.456), consistent with broader artist listening patterns.

Gain and **WinRate** are result-side summaries, not input-side behavioral indicators. Gain is the mean signed difference between RouteRec’s score and the score of the strongest per-metric baseline, averaged over all reported metrics for that dataset. WinRate is the fraction of the nine reported metric values where RouteRec ranks first among all compared methods. We include them in the same table only to make it easier to compare the dataset profile with the final performance pattern.

Table 8: Full experimental results across all cutoffs. Datasets marked with [†] are sampled processed subsets. Each method reports the run with the best mean validation performance under the shared selection rule. Lower Avg. Rank is better.

Dataset	Metric	SASRec	GRU4Rec	TiSASRec	FEARec	DuoRec	BSARec	FAME	DIF-SR	FDSA	RouteRec
KuaiRec [†]	HR@5	0.3195	0.2770	0.3004	<u>0.3397</u>	0.3270	0.3281	0.3255	0.2275	0.3270	0.3494
	HR@10	0.3284	0.3061	0.3216	<u>0.3513</u>	0.3468	0.3359	0.3307	0.2333	0.3453	0.3669
	HR@20	0.3379	0.3367	0.3429	<u>0.3688</u>	0.3677	0.3464	0.3430	0.2411	0.3654	0.3949
	NDCG@5	0.3148	0.2493	0.2820	<u>0.3249</u>	0.3152	0.3184	0.3172	0.2210	0.3140	0.3410
	NDCG@10	0.3177	0.2589	0.2889	<u>0.3286</u>	0.3215	0.3209	0.3188	0.2229	0.3197	0.3467
	NDCG@20	0.3200	0.2666	0.2942	<u>0.3330</u>	0.3269	0.3236	0.3220	0.2248	0.3248	0.3538
	MRR@5	0.3133	0.2400	0.2759	<u>0.3200</u>	0.3112	0.3152	0.3144	0.2188	0.3097	0.3382
	MRR@10	0.3144	0.2441	0.2788	<u>0.3215</u>	0.3138	0.3162	0.3151	0.2196	0.3120	0.3406
	MRR@20	0.3150	0.2462	0.2802	<u>0.3227</u>	0.3153	0.3170	0.3160	0.2201	0.3133	0.3425
Foursquare	HR@5	0.2350	0.1741	0.2388	0.2403	0.2438	0.1995	0.1954	0.2458	<u>0.2514</u>	0.2559
	HR@10	0.3049	0.2180	0.2976	0.3054	<u>0.3195</u>	0.2383	0.2417	0.3032	<u>0.3042</u>	0.3215
	HR@20	0.3635	0.2593	0.3501	0.3620	<u>0.3760</u>	0.2773	0.2800	0.3536	0.3524	0.3776
	NDCG@5	0.1703	0.1334	0.1735	0.1714	<u>0.1726</u>	0.1483	0.1455	0.1821	0.1843	0.1829
	NDCG@10	0.1929	0.1476	0.1926	0.1925	0.1970	0.1609	0.1604	0.2008	<u>0.2014</u>	0.2042
	NDCG@20	0.2078	0.1580	0.2059	0.2068	0.2114	0.1708	0.1701	0.2135	<u>0.2136</u>	0.2184
	MRR@5	0.1489	0.1199	0.1519	0.1486	0.1491	0.1313	0.1289	<u>0.1611</u>	0.1621	0.1588
	MRR@10	0.1583	0.1258	0.1599	0.1574	0.1591	0.1366	0.1351	<u>0.1688</u>	0.1692	0.1676
	MRR@20	0.1624	0.1286	0.1635	0.1613	0.1631	0.1393	0.1378	<u>0.1723</u>	0.1726	0.1715
LastFM [†]	HR@5	0.3471	0.2836	0.3405	<u>0.3630</u>	0.3521	0.3312	0.3173	0.3389	0.3494	0.3693
	HR@10	0.3775	0.3109	0.3715	<u>0.3876</u>	0.3850	0.3475	0.3359	0.3593	0.3762	0.3965
	HR@20	0.3999	0.3338	0.3997	<u>0.4103</u>	0.4101	0.3615	0.3554	0.3825	0.4020	0.4194
	NDCG@5	0.2999	0.2588	0.2929	<u>0.3108</u>	0.2959	0.2992	0.2911	0.3103	<u>0.3161</u>	0.3218
	NDCG@10	0.3097	0.2676	0.3029	0.3187	0.3066	0.3044	0.2972	0.3169	<u>0.3247</u>	0.3307
	NDCG@20	0.3154	0.2735	0.3101	0.3245	0.3129	0.3080	0.3020	0.3228	<u>0.3313</u>	0.3365
	MRR@5	0.2841	0.2506	0.2770	0.2933	0.2771	0.2884	0.2822	0.3008	<u>0.3050</u>	0.3059
	MRR@10	0.2881	0.2542	0.2812	0.2965	0.2815	0.2906	0.2847	0.3035	<u>0.3085</u>	0.3096
	MRR@20	0.2897	0.2558	0.2831	0.2981	0.2833	0.2915	0.2861	0.3051	<u>0.3103</u>	0.3113
Retail Rocket	HR@5	0.4610	0.4312	0.4433	<u>0.4730</u>	0.4711	0.4411	0.4398	0.4262	0.4579	0.4862
	HR@10	0.5321	0.5048	0.5131	<u>0.5487</u>	0.5478	0.4655	0.4612	0.4322	0.5343	0.5608
	HR@20	0.5988	0.5719	0.5810	<u>0.6234</u>	0.6194	0.4887	0.4812	0.4365	0.6053	0.6315
	NDCG@5	0.3676	0.3436	0.3528	<u>0.3752</u>	0.3745	<u>0.3877</u>	0.3874	0.3844	0.3600	0.3907
	NDCG@10	0.3907	0.3674	0.3755	<u>0.3998</u>	0.3994	<u>0.3957</u>	0.3943	0.3863	0.3848	0.4148
	NDCG@20	0.4076	0.3844	0.3926	<u>0.4187</u>	0.4175	0.4015	0.3994	0.3874	0.4028	0.4327
	MRR@5	0.3365	0.3145	0.3227	<u>0.3427</u>	0.3423	<u>0.3696</u>	0.3695	0.3698	0.3274	0.3588
	MRR@10	0.3461	0.3243	0.3321	0.3529	0.3526	0.3729	<u>0.3724</u>	0.3706	0.3377	0.3688
	MRR@20	0.3507	0.3290	0.3368	0.3581	0.3576	0.3745	<u>0.3738</u>	0.3709	0.3427	0.3737
Beauty	HR@5	0.1038	0.0573	0.0918	0.1003	0.0993	0.0630	0.0372	0.0993	<u>0.1041</u>	0.1087
	HR@10	0.1324	0.0840	0.1235	<u>0.1404</u>	0.1347	0.0726	0.0497	0.1165	<u>0.1318</u>	0.1561
	HR@20	0.1637	0.1146	0.1630	<u>0.1757</u>	<u>0.1958</u>	0.0955	0.0592	0.1385	0.1681	0.2009
	NDCG@5	0.0707	0.0372	0.0729	0.0669	<u>0.0678</u>	0.0474	0.0263	0.0756	0.0830	<u>0.0798</u>
	NDCG@10	0.0800	0.0457	0.0831	0.0797	0.0793	0.0504	0.0303	0.0811	<u>0.0918</u>	0.0948
	NDCG@20	0.0879	0.0533	0.0931	0.0889	0.0948	0.0563	0.0327	0.0867	<u>0.1011</u>	0.1062
	MRR@5	0.0598	0.0307	0.0666	0.0556	0.0571	0.0422	0.0228	0.0677	0.0761	<u>0.0701</u>
	MRR@10	0.0637	0.0341	0.0707	0.0608	0.0619	0.0434	0.0244	0.0700	0.0796	<u>0.0762</u>
	MRR@20	0.0659	0.0361	0.0735	0.0634	0.0662	0.0450	0.0250	0.0716	0.0822	<u>0.0794</u>
ML-1M	HR@5	0.0858	0.0984	0.0906	0.0906	0.0977	0.0892	0.0968	0.0384	0.1022	0.1005
	HR@10	0.1423	0.1456	0.1448	0.1490	0.1555	0.1374	0.1533	0.0669	0.1606	<u>0.1568</u>
	HR@20	0.2157	0.2181	0.2220	0.2234	<u>0.2402</u>	0.2018	0.2243	0.1052	0.2426	0.2353
	NDCG@5	0.0536	0.0626	0.0582	0.0549	<u>0.0624</u>	0.0568	<u>0.0637</u>	0.0240	0.0645	0.0627
	NDCG@10	0.0717	0.0778	0.0756	0.0736	0.0810	0.0723	<u>0.0818</u>	0.0331	0.0833	0.0807
	NDCG@20	0.0902	0.0960	0.0950	0.0923	<u>0.1022</u>	0.0884	0.0996	0.0427	0.1038	0.1004
	MRR@5	0.0431	0.0510	0.0476	0.0432	<u>0.0508</u>	0.0463	0.0529	0.0192	<u>0.0522</u>	0.0504
	MRR@10	0.0505	0.0572	0.0548	0.0508	0.0584	0.0527	0.0603	0.0230	<u>0.0600</u>	0.0577
	MRR@20	0.0555	0.0621	0.0600	0.0559	0.0641	0.0570	<u>0.0651</u>	0.0256	0.0655	0.0631
Avg. Rank↓		6.07	8.63	6.56	4.41	4.41	6.52	6.72	6.55	3.40	1.74

C.4 Directional Rank Correlations

Table 10 reports Spearman rank correlations between each candidate axis and RouteRec’s performance measures across the six datasets. Because only six datasets are compared, these values should be read as directional evidence rather than as formal statistical tests. In addition to the axes defined in Sections C.1 and C.3, we include **memory intensity** (RepeatInt), the mean repeat ratio across sessions: $\text{RepeatInt} = \text{mean}(\text{repeat_ratio})$. It captures the average level of repetition rather than its variability, and it is the axis most strongly aligned with gain.

Table 10: Spearman rank correlations between dataset-level behavioral axes and RouteRec performance across six datasets. All axis values are computed from the training split before model training.

Behavioral axis	With Gain	With WinRate
Memory intensity (RepeatInt)	0.841	0.761
Memory variability (RepeatVar)	0.667	0.851
Temporal volatility (SessVol)	0.543	0.441
Carryover	0.377	0.403
Transition branching (Branch)	0.257	−0.088
Context availability (CtxAvail)	−0.086	0.353
Popularity concentration (PopConc)	−0.143	−0.441

Memory variability and memory intensity show the highest rank correlations with gain, which fits the contrast between KuaiRec and LastFM on one side and Beauty and ML-1M on the other. In simple terms, RouteRec helps most when repetition is not only present, but also unevenly distributed across sessions, because this creates a stronger need for different routes for different behavioral regimes. Temporal volatility also shows a positive directional correlation, again matching the intuition that more irregular session dynamics create more room for behavior-guided routing.

Some axes are more subtle. Transition branching is clearly relevant in KuaiRec, but its rank correlation is weaker because high branching alone does not guarantee a gain in every dataset. Context availability shows almost no correlation with gain magnitude but a positive correlation with WinRate, which is consistent with its role in the paper: it is better read as a *supporting condition* for macro-scope routing than as a direct predictor of how large the gain should be. Popularity concentration shows negative correlations, suggesting that a head-heavy catalog by itself does not explain RouteRec’s advantage.

D Extended Framework and Structural Ablations

This section extends the main-text design-justification study with additional scope-layout controls. The goal is to test whether the selected three-scope RouteRec configuration is a meaningful architectural choice, rather than one arbitrary point in a large design space.

We compare the main three-scope configuration against several controlled alternatives. *Single-view (macro only)* keeps only the broad cross-session view. *Two-view (macro+mid)* removes the

most local router and keeps only broader cues. *Local-first ordering* removes macro and keeps only mid+micro, so local evidence enters earlier and broader evidence disappears. *Global-late ordering* removes mid and keeps macro+micro, so the broader signal returns only after the first routed decision. The common expectation across these controls is simple: if RouteRec really benefits from separating broader and narrower behavioral evidence, then removing one scope or changing the intended order should weaken that separation and usually reduce performance.

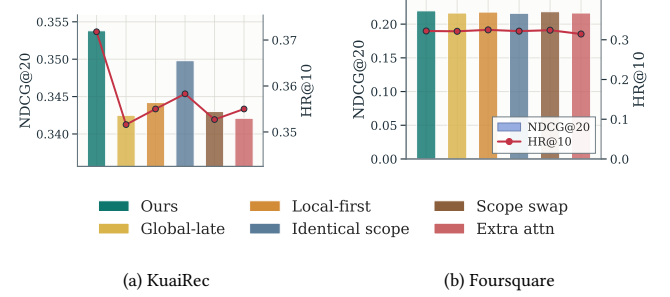


Figure 7: Additional scope-layout ablations on KuaiRec (left) and Foursquare (right). The main three-scope RouteRec configuration is compared with reduced one-scope and two-scope controls that remove either broader or narrower routing evidence, so the figure tests whether the intended coarse-to-fine scope allocation is itself responsible for the gain.

Across both panels, the main configuration remains the strongest or tied with the strongest variants, while the simpler controls usually lose some accuracy. The pattern is especially informative for the two-view variants. Removing micro (*Two-view (macro+mid)*) is often less damaging than removing macro (*Local-first ordering*), which is consistent with the main-text result that broader cross-session context is an important part of the routing signal. *Global-late ordering* is sometimes closer to the full model than *Local-first ordering*, but it still does not produce a cleaner or more reliable alternative, which suggests that simply keeping macro is not enough; it also matters where that broader evidence enters the computation.

We also inspected three additional sanity checks in the ablation code. *Identical scope* forces macro, mid, and micro to observe the same session-level range, removing the intended role separation across scopes. *Scope swap* assigns the wrong time range to at least one scope, such as forcing macro to act at the token level. *Extra attn* adds one more attention block without adding a new routing role, isolating the effect of extra backbone capacity from the effect of better scope design. These extra controls do not reveal a cleaner replacement for the main design: forcing the same observation range across scopes changes the scores only marginally, swapping scope roles produces unstable trade-offs, and adding attention alone does not explain the gain. Taken together, the evidence supports the interpretation that RouteRec benefits from *role separation across scopes*, not merely from having more blocks or more opportunities to route.

E Where RouteRec Helps Most: Special Bins

This section asks a different question from the structural ablations above. Instead of changing the model, we keep the trained model fixed and ask *where* its advantage appears within the data. We therefore partition KuaiRec by session length and by target-item frequency, then compare RouteRec against the strongest non-RouteRec baseline within each bin. KuaiRec is a useful case because it combines strong behavioral variation with a pronounced popularity structure.

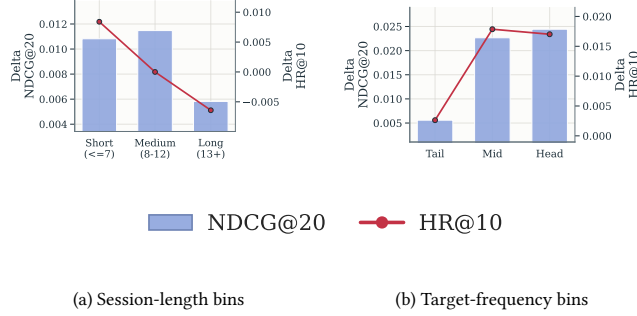


Figure 8: Within-dataset gain analysis on KuaiRec. Each bar shows RouteRec’s improvement over the strongest non-RouteRec baseline in the same bin, partitioned by session length (left) and target-item frequency (right). This view tests whether the overall gain is concentrated only in a narrow slice of the data or persists across behaviorally different conditions.

If behavior-guided routing helps mainly when the model must decide quickly from limited evidence, we would expect larger gains in shorter or medium-length contexts than in very long contexts. If the gain came only from exploiting popular targets, we would also expect the advantage to collapse in the tail. The figure does not support either narrow explanation. The strongest margin appears in shorter and medium contexts, which is consistent with the idea that routing is most useful before a long shared history is enough to dominate the prediction. At the same time, the gain remains visible across the target-frequency range, which suggests that the method is not simply a head-item trick.

F Objective and Regularization Variants

This section tests whether the selected learning objective is doing all of the work, or whether RouteRec remains strong once the routing architecture is fixed and only the objective is changed. The comparison therefore keeps the same model family and changes only the training objective or regularization terms.

The control logic is straightforward. The full RouteRec objective combines next-item supervision with auxiliary terms that stabilize routing and encourage meaningful specialization. The ablation variants remove or isolate some of those terms. If the paper’s gains came mainly from one auxiliary loss rather than from the routing design itself, we would expect performance to collapse once that one term is removed. If, instead, the auxiliary terms mainly refine an already useful architecture, then the neighboring variants should remain in the same general performance band even when the full objective stays best.

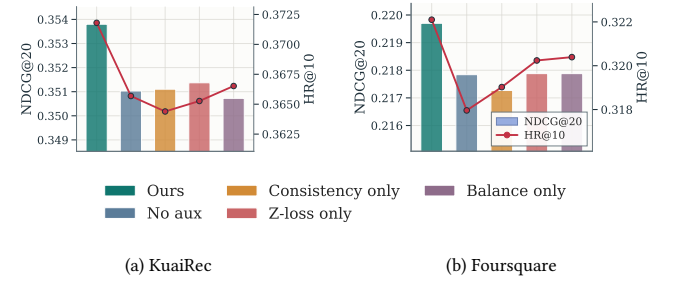


Figure 9: Objective variants on KuaiRec (left) and Foursquare (right). The full RouteRec objective is compared with simplified alternatives that remove or isolate auxiliary routing terms, so the figure tests whether the observed gains require the complete objective or remain broadly stable under nearby training choices.

The chosen objective remains the strongest overall, but the neighboring settings stay in the same general range. This is the pattern we would expect if the auxiliary terms are helpful refinements rather than the sole source of RouteRec’s advantage. In other words, the objective matters, but it does not replace the architectural story: the routing design already has value, and the full objective mainly makes that design more stable and easier to optimize.